Music Player – Advanced Web Development Project

By George Nicholl

Overview

This Music Player is a modern, browser-based audio player developed using HTML, CSS and JavaScript. It allows users to play, pause, skip, loop and shuffle songs with a clean, visually appealing interface. The player also features a dynamic playlist, progress bar seeking and a responsive layout that adopts seamlessly to different devices.

This project demonstrates an understanding of front-end web development principles including DOM manipulation, event driven programming and responsive UI design.

Through this build, I aimed to create a music player that not only functions smoothly but also provide a pleasant user experience – similar to a simplified version of Spotify or Apple Music.

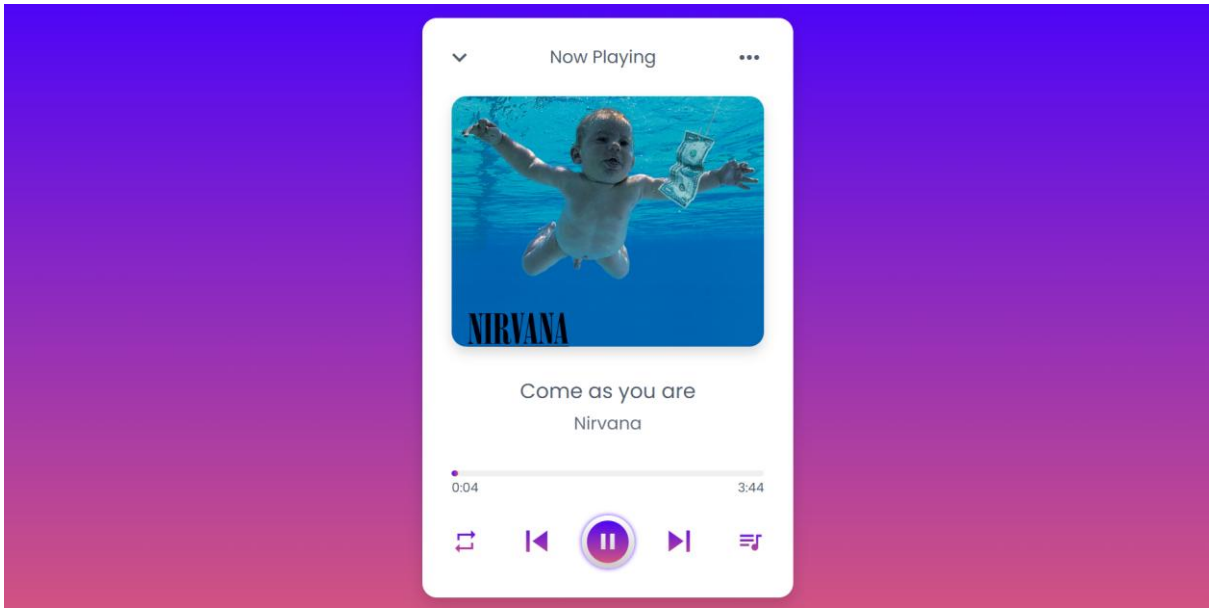

Figure 1: Final Interface of the Music Player

Technology Stack

Component	Description
Languages Used	HTML, CSS, JavaScript
Framework / Libraries	Vanilla JavaScript (no external frameworks)
Tools / IDE	Visual Studio Code, Liver Server
API's	HTML5 Audio API
Design Approach	Modular, Responsive, user-centred design
Browser Compatibility	Fully Compatible with modern browsers (Chrome, Edge, Firefox)

The decision to build this project using pure JavaScript was to strengthen understanding of native browser capabilities without reliance on frameworks.

Project Goals & Learning Outcomes

Project Goals

The aim of this project was to design and implement a fully functional and visually engaging music player that runs entirely in the browser. The focus was on mastering HTML structure, CSS styling for the layout and responsiveness, and JavaScript for interactivity and state management.

Learning Outcomes

Through this project I developed and refined skills in:

- DOM Manipulation – Dynamically changing UI Elements (song title, album art, and progress bar)
- Modular Programming – Separating HTML, CSS and JS for scalability and clarity.
- Responsive Web Design – Using Flexbox and relative sizing for adaptability.
- Audio API Control – Managing Audio playback via the built-in HTML5 Audio Api.

System Design & Architecture

This Music Player project follows a modular architecture, separating structure (HTML), presentation (CSS) and functionality (JavaScript) to ensure scalability, readability and maintainability.

File Structure

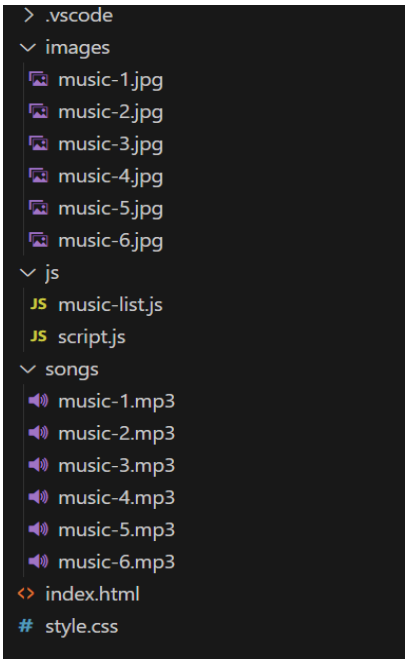

Figure 2:Project Folder Structure

Data Flow Overview

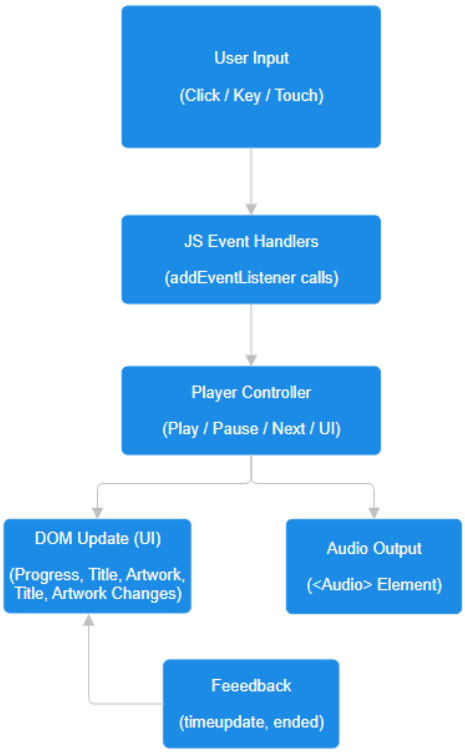

Figure 3: Data Flow Diagram of Music Player Functionality

<u>Core Components</u>

HTML Structure

The HTML file forms the backbone of the Music Player. It defines all sections and components that will later be styled and given functionality.

Key elements include:

- Top Bar: Displays “Now Playing” and includes expand/collapse icons.
- Image Area: shows the current songs album art.
- Song Details: Displays artist and song name dynamically.
- Progress Area: Contains the progress bar and time indicators (current/total).
- Controls: Includes play, pause, next, previous, shuffle and repeat icons.
- Music List Selection: Hidden by default but displayed when toggled by the user.

Each section is wrapped inside a container, ensuring a centralised and responsive layout.

index.html

index.html > html > body > div.wrapper > div.progress-area

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Music Player</title>
  <link rel="stylesheet" href="style.css">
  <link rel="stylesheet" href="https://fonts.googleapis.com/icon?family=Material+Icons">
</head>
<body>
  <div class="wrapper">
    <div class="top-bar">
      <i class="material-icons">expand_more</i>
      <span>Now Playing</span>
      <i class="material-icons">more_horiz</i>
    </div>
    <div class="img-area">
      <img src="" alt="">
    </div>
    <div class="song-details">
      <p class="name"></p>
      <p class="artist"></p>
    </div>
    <div class="progress-area">
      <div class="progress-bar">
        <audio id="main-audio" src=""></audio>
      </div>
      <div class="song-timer">
        <span class="current-time">0:00</span>
        <span class="max-duration">0:00</span>
      </div>
    </div>
    <div class="controls">
      <i id="repeat-plist" class="material-icons" title="Playlist looped">repeat</i>
      <i id="prev" class="material-icons">skip_previous</i>
      <div class="play-pause">
        <i class="material-icons play">play_arrow</i>
      </div>
      <i id="next" class="material-icons">skip_next</i>
      <i id="more-music" class="material-icons">queue_music</i>
    </div>
    <div class="music-list">
      <div class="header">
        <div class="row">
          <i class= "list material-icons">queue_music</i>
          <span>Music list</span>
        </div>
        <i id="close" class="material-icons">close</i>
      </div>
      <ul>
        <!-- here li list are coming from js -->
      </ul>
    </div>
  </div>

  <script src="js/music-list.js"></script>
  <script src="js/script.js"></script>

</body>
</html>
```

Figure 4: HTML Base structure of the Music Player

The HTML uses semantic and descriptive classes to make it easier for CSS and JS to target and manipulate specific elements.

CSS Styling

The CSS stylesheet is responsible for the visual presentation for the player. The overall design is modern and minimalist, with a pink and violet gradient theme set against a clean white player container.

Key styling features.

- Global Reset – Removes default browser margins and padding to maintain layout consistency.
- Root Variables – Define consistent colour themes (e.g –purple, pink) for easy modification.
- Flexbox Layout – Centres and aligns content both vertically and horizontally.
- Wrapper Styling – Applies padding, rounded corners, and shadow for a polished appearance.
- Component Styling – Each section (controls, song details, progress bar) is styled for clarity and usability.
- Responsive Design – Uses media queries to adapt the layout for smaller screens such as phones or tablets.

```
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@200;300;400;500;600;700&display=swap');
*{
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Poppins", sans-serif;
}
*::before, *::after{
  padding: 0;
  margin: 0;
}
:root{
  --purple: #4d04f8;
  --violet: #d25480;
  --lightblack: #515C6F;
  --white: #ffffff;
  --darkwhite: #cecaca;
  --pinkshadow: #ffcbdd;
  --lightbshadow: rgba(0,0,0,0.15);
```

Figure 5: Global Reset & Root Variables used for theming

```
body{
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 100vh;
  background: linear-gradient(var(--purple) 0%, var(--violet) 100%);
}
```

Figure 6: Body Styling showing flexbox centering and gradient background

```
.wrapper{
  width: 380px;
  padding: 25px 30px;
  overflow: hidden;
  position: relative;
  border-radius: 15px;
  background: var(--white);
  box-shadow: 0px 6px 15px var(--lightbshadow);
```

Figure 7: Wrapper Container styling creating the card-style player

```
.progress-area{
  height: 6px;
  width: 100%;
  border-radius: 50px;
  background: #f0f0f0;
  cursor: pointer;
}
.progress-area .progress-bar{
  height: inherit;
  width: 0%;
  position: relative;
  border-radius: inherit;
  background: linear-gradient(90deg, var(--purple) 0%, var(--violet) 100%);
}
.progress-bar::before{
  content: "";
  position: absolute;
  height: 12px;
  width: 12px;
  border-radius: 50%;
  top: 50%;
  right: -5px;
  z-index: 2;
  opacity: 0;
  pointer-events: none;
  transform: translateY(-50%);
  background: inherit;
  transition: opacity 0.2s ease;
}
.progress-area:hover .progress-bar::before{
  opacity: 1;
  pointer-events: auto;
}
.progress-area .song-timer{
  margin-top: 2px;
```

Figure 8: Progress Bar styling with gradient fill and interactive hover knob

```
.controls{
  margin: 40px 0 5px 0;
}
.controls i{
  font-size: 28px;
  user-select: none;
  background: linear-gradient(var(--purple) 0%, var(--violet) 100%);
  background-clip: text;
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}
.controls i:nth-child(2),
.controls i:nth-child(4){
  font-size: 43px;
}
.controls #prev{
  margin-right: -13px;
}
.controls #next{
  margin-left: -13px;
}
.controls .play-pause{
  height: 54px;
  width: 54px;
  display: flex;
  cursor: pointer;
  align-items: center;
  justify-content: center;
  border-radius: 50%;
  background: linear-gradient(var(--white) 0%, var(--darkwhite) 100%);
  box-shadow: 0px 0px 5px var(--purple);
```

Figure 9: Styling for play/pause controls using gradient icon effects

```
.music-list{
  position: absolute;
  background: var(--white);
  width: 100%;
  left: 0;
  bottom: -55%;
  opacity: 0;
  pointer-events: none;
  z-index: 5;
  padding: 15px 30px;
  border-radius: 15px;
  box-shadow: 0px -5px 10px rgba(0,0,0,0.1);
  transition: all 0.15s ease-out;
}
.music-list.show{
  bottom: 0;
  opacity: 1;
  pointer-events: auto;
```

Figure 10: Slide-Up Playlist styling and transition effects

JavaScript Functionality

The Javascript provides interactivity and logic, controlling the behaviour of the music player through two main files:

1. Music-list.js
2. Script.js

<u>Music-list.js</u>

This script defines the array of songs, each represented as an object with four properties:

- Name – the song title
- Img – Path to the album art image
- Src – path to the audio file

This structure allows for easy addition or removal of songs by simply modifying the array.

```
js > JS music-list.js > allMusic
let allMusic = [
  {
    name: "Creep",
    artist: "Radiohead",
    img: "music-1",
    src: "music-1"
  },
  {
    name: "Come as you are",
    artist: "Nirvana",
    img: "music-2",
    src: "music-2"
  },
  {
    name: "Jump",
    artist: "Van Halen",
    img: "music-3",
    src: "music-3"
  },
  {
    name: "If I can Dream",
    artist: "Elvis Presley",
    img: "music-4",
    src: "music-4"
  },
  {
    name: "Tiny Dancer",
    artist: "Elton John",
    img: "music-5",
    src: "music-5"
  },
  {
    name: "Starman",
    artist: "David Bowie",
    img: "music-6",
    src: "music-6"
  },
];
```

Figure 11: JavaScript data structure defining the playlist using an array of song objects

Script.js

The main logic for the player resides in this file.

Initialisation:

Varibales are declared using querySelector() to reference HTML elements such as buttons, song details, and progress bar.

A variable musicindex keeps track of which song is currently active, while isMusicPaused tracks the play state.

```
//show music list
moreMusicBtn.addEventListener("click", ()=>{
  musicList.classList.toggle("show");
});
closemoreMusic.addEventListener("click", ()=>{
  moreMusicBtn.click();
});

const ulTag = wrapper.querySelector("ul");
for (let i = 0; i < allMusic.length; i++) {
  let liTag = `<li li-index="${i + 1}">
                <div class="row">
                  <span>${allMusic[i].name}</span>
                  <p>${allMusic[i].artist}</p>
                </div>
                <span id="${allMusic[i].src}" class="audio-duration">3:40</span>
                <audio class="${allMusic[i].src}" src="songs/${allMusic[i].src}.mp3"></audio>
              </li>`;
  ulTag.insertAdjacentHTML("beforeend", liTag);

  let liAudioDuartionTag = ulTag.querySelector(`#${allMusic[i].src}`);
  let liAudioTag = ulTag.querySelector(`.${allMusic[i].src}`);
  liAudioTag.addEventListener("loadeddata", ()=>{
    let duration = liAudioTag.duration;
    let totalMin = Math.floor(duration / 60);
    let totalSec = Math.floor(duration % 60);
    if(totalSec < 10){
      totalSec = `0${totalSec}`;
    };
    liAudioDuartionTag.innerText = `${totalMin}:${totalSec}`;
    liAudioDuartionTag.setAttribute("t-duration", `${totalMin}:${totalSec}`);
  });
}
```

Figure 12: Initialisation of DOM references and core state variables in script.js

Load Music:

The loadMusic() function dynamically loads the song details (name, artist, image, audio source) from the allMusic array based on the current index. It is called both when the page loads and when songs are changed.

```
window.addEventListener("load", ()=>{
    loadMusic(musicIndex);
    playingSong();
  });

function loadMusic(indexNumb){
    musicName.innerText = allMusic[indexNumb - 1].name;
    musicArtist.innerText = allMusic[indexNumb - 1].artist;
    musicImg.src = `images/${allMusic[indexNumb - 1].src}.jpg`;
    mainAudio.src = `songs/${allMusic[indexNumb - 1].src}.mp3`;
  }
```

Figure 13: loadMusic() function loading the current song details from the playlist

Play and Pause

The playMusic() and pauseMusic() functions toggle between play and pause states. When play is triggered, the class paused is added to the wrapper to change the icon and start audio playback. When paused, the class is removed and playback is stopped.

```
//play music
function playMusic(){
  wrapper.classList.add("paused");
  playPauseBtn.querySelector("i").innerText = "pause";
  mainAudio.play();
}

//pause music
function pauseMusic(){
  wrapper.classList.remove("paused");
  playPauseBtn.querySelector("i").innerText = "play_arrow";
  mainAudio.pause();
}
```

Figure 14: Playback Control functions for playing and pausing audio

Next and Previous songs

The nextMusic() and prevMusic() functions allow seamless navigation through the playlist. They increment or decrement the index and reload the appropriate song, looping back to the start or the end of the playlist when needed.

```
//prev music function
function prevMusic(){
  musicIndex--;
  musicIndex < 1 ? musicIndex = allMusic.length : musicIndex = musicIndex;
  loadMusic(musicIndex);
  playMusic();
  playingSong();
}

//next music function
function nextMusic(){
  musicIndex++;
  musicIndex > allMusic.length ? musicIndex = 1 : musicIndex = musicIndex;
  loadMusic(musicIndex);
  playMusic();
  playingSong();
}
```

Figure 15: Track navigation logic for next and previous songs with wrap-around

Progress Bar and Time Updates

The timeupdate event listener updates the progress bar width according to playback progress. It also dynamically displays elapsed and total time in mm:ss format

```
// update progress bar
mainAudio.addEventListener("timeupdate", (e)=>{
  const currentTime = e.target.currentTime;
  const duration = e.target.duration;
  let progressWidth = (currentTime / duration) * 100;
  progressBar.style.width = `${progressWidth}%`;

  let musicCurrentTime = wrapper.querySelector(".current-time"),
  musicDuartion = wrapper.querySelector(".max-duration");
  mainAudio.addEventListener("loadeddata", ()=>{
    // update song total duration
    let mainAdDuration = mainAudio.duration;
    let totalMin = Math.floor(mainAdDuration / 60);
    let totalSec = Math.floor(mainAdDuration % 60);
    if(totalSec < 10){
      totalSec = `0${totalSec}`;
    }
    musicDuartion.innerText = `${totalMin}:${totalSec}`;
  });
  // update playing song current time
  let currentMin = Math.floor(currentTime / 60);
  let currentSec = Math.floor(currentTime % 60);
  if(currentSec < 10){
    currentSec = `0${currentSec}`;
  }
  musicCurrentTime.innerText = `${currentMin}:${currentSec}`;
});
```

Figure 16: Progress bar and timer updates driven by the timeupdate event

Seek Functionality

Users can click anywhere on the progress bar to jump to that point in the song. This achieved by calculating the click position relative to the bar width and updating the audio's currentTime.

```
// update playing song currentTime on according to the progress bar width
progressArea.addEventListener("click", (e)=>{
  let progressWidth = progressArea.clientWidth;
  let clickedOffsetX = e.offsetX;
  let songDuration = mainAudio.duration;

  mainAudio.currentTime = (clickedOffsetX / progressWidth) * songDuration;
  playMusic();
  playingSong();
});
```

Figure 17: Seek functionality allowing users to jump to any point in the track

Looping and Shuffle

The loop button cycles between three modes, normal, repeat-one and shuffle. Each mode pdates the button text and behaviour:

- Repeat: Plays entire playlist again after last song.
- Repeat One: Replays the same track when it ends.
- Shuffle: Randomly selects the next track.

```
//change loop, shuffle, repeat icon onclick
const repeatBtn = wrapper.querySelector("#repeat-plist");
repeatBtn.addEventListener("click", ()=>{
  let getText = repeatBtn.innerText;
  switch(getText){
    case "repeat":
      repeatBtn.innerText = "repeat_one";
      repeatBtn.setAttribute("title", "Song looped");
      break;
    case "repeat_one":
      repeatBtn.innerText = "shuffle";
      repeatBtn.setAttribute("title", "Playback shuffled");
      break;
    case "shuffle":
      repeatBtn.innerText = "repeat";
      repeatBtn.setAttribute("title", "Playlist looped");
      break;
  }
});
```

Figure 18: Loop and shuffle mode logic handling normal, repeat-one and shuffle playback

Music List and Selections

The playlist menu is dynamically generated from the allMusic array. When the user opens the list, they can click on any song title to play it immediately. The clicked item updates the index and triggers loadMusic() and playMusic().

```
.music-list ul li{
  list-style: none;
  display: flex;
  cursor: pointer;
  padding-bottom: 10px;
  margin-bottom: 5px;
  color: var(--lightblack);
  border-bottom: 1px solid #E5E5E5;
}
.music-list ul li:last-child{
  border-bottom: 0px;
}
.music-list ul li .row span{
  font-size: 17px;
}
.music-list ul li .row p{
  opacity: 0.9;
}
ul li .audio-duration{
  font-size: 16px;
}
ul li.playing{
  pointer-events: none;
  color: var(--purple);
}
```

Figure 19: Dynamic playlist generation and song selection handlers

Testing and Debugging

Extensive testing was carried out to ensure smooth functionality and responsive UI performance.

Test Case	Expected Result	Outcome
Play button clicked	Song Starts Playing	Successful ☑
Pause Button clicked	Song Pauses	Successful ☑
Next/Previous pressed	Correct song loads	Successful ☑
Progress bar Clicked	Song seeks to chosen time	Successful ☑
Shuffle enabled	Next track chosen randomly	Successful ☑
Loop One enabled	Same song repeats	Successful ☑
Playlist Opened	Displays all songs correctly	Successful ☑

Debugging Techniques Used

- Console Logging – Used to verify variable updates and event triggers.
- Stepwise Testing – Each feature (play, skip, progress) tested individually before combining
- Responsive Layout Testing – Verified using Chrome DevTools and manual resizing.
- Error Handling – Checked for missing files or undefined variables.

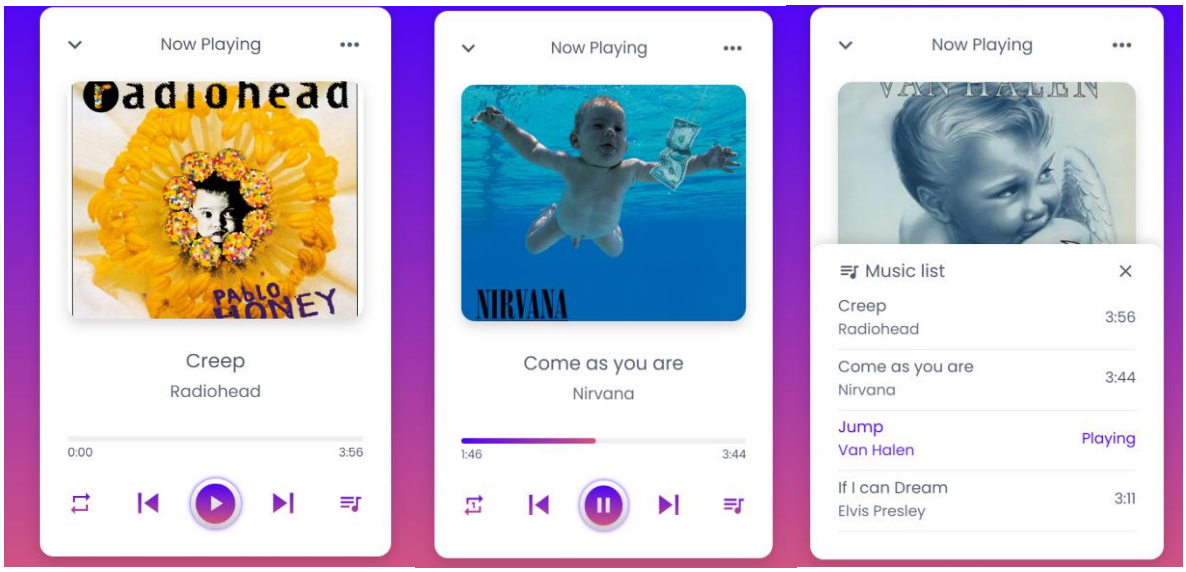

Figure 20: Testing all player states

Challenges and Solutions

Challenge	Solution
Synchronising UI with playback state	Used event listeners (timeupdate, ended) to trigger UI updates in real time.
Managing multiple playback modes	Implemented a loop toggle logic with controlled conditions
Keeping layout responsive	Applied flexible units and media queries to the wrapper container.
Preventing code duplication	Split logic between script.js (functions) and music-list.js (data)

Future Improvements

Planned future enhancements include:

- Volume control slider and mute toggle
- Saving playlist position and preferences with localStorage
- Adding visual animations or waveform visualisation
- Implementing playlist sorting or drag-and-drop reordering
- Introducing dark and light mode themes.

Conclusion

The music Player effectively demonstrates the integration of HTML, CSS and JavaScript to create a functional and visually appealing web application. Through modular structure, event-driven interactivity and responsive design, it replicates essential features of modern digital media players.

This project strengthened my ability to:

- Design clean and maintainable web structures.
- Implement real-time UI updates with JavaScript.
- Manage application states effectively.

Overall, this project showcases a blend of technical skill and creative design, forming a solid foundation for future work in front-end development and interactive web applications.